

# A Comprehensive Framework for EUR/USD Prediction: From Adaptive Labeling to Regime-Aware Model Evaluation

*A Causal, Event-Driven Machine-Learning Pipeline for Optimal Trade Entry Detection on 5-Minute Foreign-Exchange Data*

Quantitative Machine-Learning Research — Smart Money / OTE Pipeline  
Technical Research Report • Prepared for the Financial Data Science Community

---

## Abstract

This paper documents a complete, production-oriented machine-learning pipeline for detecting Optimal Trade Entry (OTE) zones on the EUR/USD 5-minute foreign-exchange series. The system is engineered around a single difficult question evaluated causally at each bar close: is the current bar inside a high-quality long or short optimal-entry zone, and with what confidence? Rather than treating direction prediction as a generic per-bar classification task, the framework reframes it as rare-event detection and builds every stage — labeling, feature engineering, feature selection, model training, and evaluation — to respect causality, volatility scaling, temporal hygiene, and the economics of trading frictions. Labels are produced by three causal sub-engines (reversal, continuation-pullback, and breakout) whose thresholds are expressed in structural Average True Range (ATR) units, validated by a triple-barrier test (take-profit  $1.0\times$  ATR, stop  $2.0\times$  ATR, 120-bar vertical barrier for reversals), cross-checked by a forward trend-scan t-statistic, quality-scored on a four-component rubric, and unioned into synthetic `long_ote` / `short_ote` targets with explicit exclusion masks and concurrency-aware sample weights. Feature generation assembles roughly 1,600 engineered columns spanning price action, volatility, trend, momentum, structure, higher-timeframe context, Inner-Circle-Trader (ICT) concepts, microstructure proxies, fractional differentiation, session/calendar state, and an optional catalogue of 400+ rule-based strategy signals. Selection proceeds in two layers: a composite base ranking (absolute association, mutual information, random-forest importance) followed by a backend-aware attribution stage that re-ranks features with TreeSHAP for XGBoost and integrated gradients for the sequence models, then applies floor and cumulative-importance gates. Three model families (XGBoost on sparse lag windows; Temporal Convolutional Networks and LSTMs on dense causal windows) are trained under purged, expanding-window walk-forward cross-validation with focal loss, hard-negative up-weighting, probability calibration, and an event-level threshold search. Evaluation is regime-aware: bars are tagged with a pairwise trend $\times$ volatility composite regime, model skill is sliced within each regime with bootstrapped average-precision confidence intervals, prediction thresholds are calibrated per regime against a cost-aware, leave-one-year-out objective, and a final 49-fold walk-forward backtest with explicit spread, slippage, and commission frictions measures post-cost expectancy, profit factor, Sharpe, drawdown, and walk-forward efficiency. On the held-out comparison cohort the TCN

backend dominates: long\_ote TCN reaches a test average precision (AP) of 0.763 and event F0.5 of 0.852, winning all 27 evaluated regime slices, while short\_ote TCN reaches AP 0.738 / event F0.5 0.787. In the cost-aware walk-forward backtest the long TCN delivers ≈53,400 net pips over 13.5 years at a profit factor of 1.96 and monthly Sharpe 2.36, and the short TCN ≈34,100 net pips at profit factor 1.70 / Sharpe 1.90. We report where the framework works (trend-aligned, volatile, London/overlap sessions), where it breaks down (thin counter-trend buckets, low-volatility ranges), and the negative result that the current hard-abstain policy, while sharply raising event precision, suppresses too much throughput to be profitable.

## 1. Introduction

Predicting short-horizon movements in a liquid currency pair such as EUR/USD is among the least forgiving problems in applied machine learning. The signal-to-noise ratio is extreme: 5-minute log returns are near-martingale, dominated by microstructure noise, and only intermittently organized into the structural moves a discretionary trader would consider tradeable. The data-generating process is non-stationary across volatility regimes, monetary-policy cycles, and liquidity sessions; a relationship that holds during a trending London session can invert during an Asian-session range. Worse, the conventional machine-learning workflow is riddled with traps that flatter offline metrics while destroying live performance: random train/test shuffling leaks the future into the past, overlapping labels inflate effective sample size, fixed-pip thresholds become meaningless as volatility expands and contracts, and a model tuned for raw accuracy learns to predict the overwhelmingly common 'do nothing' class.

These failure modes explain a persistent gap between academic financial ML and practical trading-system design. The academic literature frequently optimizes a single classifier for bar-level accuracy or AUC on a shuffled split; the practitioner needs a system that emits sparse, high-precision, economically meaningful signals, survives realistic transaction costs, and behaves predictably across regimes. The framework documented here is built explicitly for the second world. It is not a single classifier but an integrated pipeline whose design principles are encoded directly in code and configuration:

- **Causality first.** detection logic uses only information available up to the current bar; future information is confined to label validation and training-target construction.
- **Volatility awareness.** every structural threshold is expressed in structural-ATR units rather than fixed pips, so the same rule remains meaningful across volatility regimes.
- **Event-based thinking.** model selection, thresholding, and backtesting prioritize de-duplicated event detection over per-bar accuracy, matching how a trader actually consumes signals.
- **Temporal hygiene.** splits are chronological, cross-validation folds are purged with an embargo, and all scaling and imputation statistics are fit on training windows only.
- **Rare-event robustness.** focal loss, concurrency-aware sample weights, hard-negative up-weighting, and balanced tuning subsamples address the extreme class imbalance of a ~5% positive rate.
- **Structural context.** higher-timeframe swings, ICT structure, and session features are built directly into the feature space, and evaluation is conditioned on a pairwise market regime.

The contributions of this work are fourfold. First, we present an adaptive, multi-engine labeling methodology that fuses ideas from López de Prado's *Advances in Financial Machine Learning* (CUSUM event sampling, the triple-barrier method, purging, and sample uniqueness) with a structural-ATR zigzag detector and a trend-scan validator, producing labels that reflect tradeable structure rather than micro-noise. Second, we describe a two-stage feature-selection pipeline whose second stage is backend-aware — the tree model and the sequence models receive feature rankings derived from their own attribution method. Third, we train and compare three model families under identical purged walk-forward protocols, and analyse how hyperparameter and architectural choices move event-level metrics. Fourth, and most importantly, we introduce a regime-aware evaluation layer built on pairwise trend×volatility regimes, regime-conditional threshold calibration, and a cost-aware walk-forward backtest, and we use it to characterize precisely when the models work and when they fail. Throughout, we ground every claim in the actual functions, parameter values, and logged artifacts of the implementation.

## 2. Data and Preprocessing

The raw input is EUR/USD intraday OHLCV data sourced from FirstRate Data, available at 1-, 5-, and 30-minute, 1-hour, and 1-day resolutions. The implemented OTE training path requires only the 5-minute series as its base input; the 30-minute and 1-hour frames used for structural context are derived internally by resampling (first-open, max-high, min-low, last-close, summed-volume). The canonical modeling window for the full training run spans 2019-01-01 through 2026-03-25, yielding 543,441 cleaned 5-minute bars (from a 1.45M-row raw 5-minute source), 90,813 resampled 30-minute bars, and 45,425 resampled 1-hour bars.

A strict timezone contract is enforced and persisted to metadata sidecars at every stage. The raw feed is treated as GMT-6, normalized to a canonical UTC store, while the session/feature clock uses America/New\_York and the FX market-close boundary is 17:00 America/New\_York (with the trading week closing on the W-FRI boundary). Timestamps are localized with explicit daylight-saving handling, the frame is sorted and deduplicated, and missing volume is backfilled with zeros.

Before any label construction, the loader applies a deterministic anomaly filter (`detect_anomalies`). Bars are dropped when they exhibit (i) impossible OHLC geometry, (ii) a bar range exceeding  $10\times$  a rolling 20-period ATR baseline, or (iii) non-positive volume. This matters because the downstream pipeline relies heavily on local windows: a single malformed bar can contaminate many subsequent feature windows, not just one sample. On the full run, only 70 anomaly bars were removed, confirming the underlying feed quality while still guarding against pathological records. Two distinct preprocessing notions appear in the codebase and should not be confused: a lightweight base-data cleaning step (sampling, hash-based duplicate-feature removal, distribution-aware imputation), and the target-aware `FeaturePreprocessingPipeline` that produces model-ready datasets — the latter is described in Section 5. Technical indicators draw on TA-Lib and the `ta` library; the optional rule-based strategy catalogue is adapted from the `QuantifiedStrategies` corpus.

### 3. Target Labeling Methodology

The labeling layer is the conceptual heart of the framework and is deliberately not a naive 'mark every local extreme' routine. It is a causal, volatility-aware, outcome-validated pipeline orchestrated by `labeling_engine.py`, which runs three independent sub-engines in one pass and writes a single bar-level table with separately named label columns per family. The three families answer three distinct trading questions:

- **Reversal:** does the current bar sit inside a validated reversal zone at a swing extreme? (`reversal_labeling_engine.py`)
- **Continuation-pullback:** is the current bar a shallow, trend-aligned pullback within an established trend that is about to resume? (`ote_continuation_pullback_labeling_engine.py`)
- **Breakout:** is the current bar a confirmed break of a prior compressed range? (`ote_breakout_labeling_engine.py`)

Each family produces both a zone label (bars inside a validated OTE region) and a precise entry label (the single best executable bar near the validated structure). The downstream OTE modeling targets, `long_ote` and `short_ote`, are then formed as the row-wise union of the three families (Section 3.7).

#### 3.1 Why this design over standard alternatives

Three standard labeling schemes were rejected on principled grounds. A fixed-horizon directional label (sign of the  $n$ -bar-ahead return) ignores path dependence and labels noise as signal whenever the horizon straddles a reversal. A pure triple-barrier label applied to every bar produces dense, highly overlapping labels with low sample uniqueness and no notion of structural quality. A simple zigzag on raw price uses a fixed pip or percentage threshold that is either too tight in high volatility (clustered false swings) or too loose in low volatility (missed structure). The implemented approach keeps the most useful piece of each — the triple-barrier as an outcome validator (not the sampler), the zigzag as the structural detector — but anchors the detector in higher-timeframe structural ATR and layers on quality scoring, exclusion masks, and concurrency weighting. The result is a sparse, high-quality, regime-robust label set.

#### 3.2 Structural ATR and volatility normalization

The single most important design choice is that swing thresholds are scaled by a structural ATR computed on a higher timeframe rather than by the local 5-minute ATR. The reversal engine computes a 14-period Wilder (EMA) ATR on 1-hour bars and maps it causally onto the 5-minute index by forward-fill (`map_htf_atr_to_ltf`), so that a 5-minute bar always sees the most recently completed hourly volatility estimate. This lifts thresholds from the  $\sim 1$ -pip scale of 5-minute ATR to the 20–30-pip scale of hourly ATR, which is the scale at which EUR/USD structure is actually meaningful. The continuation and breakout engines use a 30-minute structural ATR by default. A reversal confirmation threshold of  $0.8\times$  structural ATR therefore stays economically meaningful whether the market is quiet or violent, because it tracks volatility instead of being fixed in price units.

#### 3.3 Causal swing detection (the zigzag gatekeepers)

`detect_swings_zigzag` is a strictly causal, alternating high/low detector. Starting after a 50-bar warmup, it tracks a running candidate extreme and confirms a swing only when price retraces against it by at least `confirm_atr_mult`  $\times$  structural ATR (using the close when `confirm_use_close` is

true). A candidate swing is accepted as a labeled swing only if it clears four simultaneous gatekeepers, which together suppress consolidation noise:

| Gatekeeper                                                      | Reversal default                    | Role                                                   |
|-----------------------------------------------------------------|-------------------------------------|--------------------------------------------------------|
| <b>Confirmation retrace</b><br>(confirm_atr_mult)               | $0.8 \times \text{structural ATR}$  | Price must reverse by this much to confirm a swing     |
| <b>Minimum swing size</b> (min_swing_atr)                       | $1.35 \times \text{structural ATR}$ | Swing must span at least this distance vs. prior swing |
| <b>Minimum price separation</b><br>(min_swing_distance_atr)     | $0.75 \times \text{structural ATR}$ | Consecutive swings must differ by this much in price   |
| <b>Minimum temporal separation</b><br>(min_bars_between_swings) | 18 bars (90 min)                    | Consecutive swings must be this far apart in time      |

Table 1. The four zigzag gatekeepers in the reversal engine. The same machinery, with looser thresholds, runs on 30-minute and 1-hour bars to produce higher-timeframe swing context.

The detector is run not only on 5-minute bars but also on the resampled 30-minute and 1-hour frames (with proportionally relaxed thresholds, e.g. htf\_confirm\_atr\_mult = 0.6, htf\_min\_swing\_atr = 0.8). Each 5-minute swing is then annotated with whether a same-direction higher-timeframe swing exists within a configurable confluence window ( $\pm 120$  minutes for 30-minute,  $\pm 240$  minutes for 1-hour), and with the number of bars since the previous same-type higher-timeframe swing. These produce both per-event validation flags (htf\_match\_30m, htf\_match\_1h) and bar-level confluence features.

### 3.4 Triple-barrier outcome validation

Detected swings are not accepted blindly. Each is validated by a triple-barrier test (validate\_swings\_triple\_barrier) applied forward from the swing confirmation index. For a swing low the entry is the confirmation close, the take-profit is set at  $+tb\_profit\_atr \times ATR$ , the stop at  $-tb\_stop\_atr \times ATR$ , and a vertical (time) barrier closes the trade after tb\_max\_bars; signs invert for a swing high. The realized outcome is recorded as tp, sl, timeout, or skip, together with the realized return and bars held. This stage filters out structurally plausible but economically weak turns. The defaults differ by family, reflecting the different risk geometry of each setup:

| Family                       | Take-profit      | Stop             | Vertical barrier | Realized win-rate (tp) |
|------------------------------|------------------|------------------|------------------|------------------------|
| <b>Reversal</b>              | $1.0 \times ATR$ | $2.0 \times ATR$ | 120 bars (10 h)  | 56.4%                  |
| <b>Continuation-pullback</b> | $0.8 \times ATR$ | $0.7 \times ATR$ | 36 bars (3 h)    | 40.8%                  |
| <b>Breakout</b>              | $0.9 \times ATR$ | $0.7 \times ATR$ | 24 bars (2 h)    | 33.7%                  |

Table 2. Triple-barrier configuration and realized take-profit hit-rate per label family (win-rates from the full-run labeling metadata). The reversal engine uses an asymmetric 1:2 profit:stop ratio with a long vertical barrier; breakouts use tighter, faster barriers.

### 3.4.1 Family-specific detection: continuation-pullback and breakout

The continuation and breakout engines subclass `ReversalParams` but implement materially different detection primitives, reflecting the distinct geometry of each setup. The continuation-pullback engine (`build_continuation_pullback_labels`) does not detect new structure de novo; it reuses the base zigzag to obtain raw 5-minute swings and then filters them into trend-aligned pullbacks (`filter_continuation_candidates`). It first builds a trend context from 30-minute and 1-hour EMAs (fast 21, slow 50): a regime is 'up' only when the fast EMA exceeds the slow EMA by at least `min_trend_spread_atr` (0.10 on 30-minute) in ATR units, and by default the 1-hour trend must agree (`require_1h_trend_alignment = true`). A swing low is accepted as a long pullback only if every one of five conditions holds: the retrace depth from the prior swing high lies in `[0.18, 2.00]` ATR (shallow enough to be a pullback, not a reversal); the pullback spans at most 48 bars; the swing sits no more than 0.50 ATR below the slow EMA 'trend floor'; the new low does not undercut the prior same-side low by more than 0.40 ATR (so it is a higher low, not a structure break); and a resumption bounce of at least 0.10 ATR is already visible by the confirmation bar. High swings mirror this for short pullbacks. In words, a continuation label marks a shallow, trend-respecting retrace that has begun to resume.

The breakout engine (`build_breakout_labels`) detects events directly from compression-then-expansion geometry (`detect_breakout_events`). Using strictly shifted rolling windows, it tracks a prior 20-bar range and a 12-bar compression range, both in ATR units. A long breakout fires when, after the bar started inside the range, price both pierces and closes above the prior range high by a 0.05-ATR buffer; the prior range is wide enough ( $\geq 0.50$  ATR) and was compressed (compression range  $\leq 1.80$  ATR, when `breakout_requires_compression` is set); the close does not over-extend past the level by more than 0.80 ATR (to avoid chasing); and the breaking candle has a body fraction  $\geq 0.25$  and a close located in the top 65% of its range. Short breakouts mirror this through the range low. The breakout entry scorer is correspondingly bespoke, replacing the reversal's wick/proximity terms with a retest-distance score and a 'hold' gate that requires price not to fall back through the broken level. These three engines thus cover the canonical trio of intraday setups — mean-reverting turns, trend continuations, and range breaks — under one causal, ATR-normalized, triple-barrier-validated framework, and their union becomes the OTE target.

## 3.5 Trend-scan cross-validation and quality scoring

Each surviving swing is further cross-validated by a forward trend-scan (`trend_scan_swings`). The engine fits a linear regression of log price over several forward windows — (12, 24, 48, 96) bars for reversals — and computes the t-statistic of the slope; the value is signed by the swing direction (positive for lows, negated for highs). The best signed t-statistic is retained, and a swing passes only if it reaches a minimum absolute t of 2.0. Formally, with slope estimate  $\beta$  and its standard error  $se(\beta)$ :

$$t = \beta / se(\beta), \quad \text{swing passes} \Leftrightarrow \text{sign}(\text{direction}) \cdot \max\_window \ t \geq 2.0$$

Every validated swing then receives a continuous quality score in `[0,1]` (`compute_swing_quality`) blending four normalized components, and is bucketed into a tier:

$$\text{quality} = 0.35 \cdot \text{tb\_score} + 0.30 \cdot \text{trend\_score} + 0.20 \cdot \text{htf\_score} + 0.15 \cdot \text{entry\_score}$$

Here `tb_score` maps the triple-barrier outcome to `{tp:1.0, timeout:0.45, skip:0.35, sl:0.0}`; `trend_score` is `min(max(t,0)/4, 1)`; `htf_score` is `0.5·1[htf_match_30m] + 0.5·1[htf_match_1h]`; and `entry_score` is the normalized entry quality from Section 3.6. Tiers are assigned `A ≥ 0.80`, `B ≥ 0.65`, `C ≥ min_label_quality` (0.35 for reversals, 0.30 continuation, 0.25 breakout), otherwise reject. A swing labelled `sl` by the triple barrier, or failing the trend-scan, is excluded from positive labeling regardless of its score. In the labeling diagnostic output the mean label quality is  $\approx 0.89$  on both sides, roughly 53% of positives reach tier A, and 86.3% of swings pass the trend-scan filter — evidence that the validation cascade is selective rather than cosmetic.

### 3.6 Zone labels, precise entry labels, exclusions, and weights

Two label granularities are produced. Zone labels (`create_zone_labels`) mark a small band around each validated swing extreme — by default 2 bars before and 0 after for reversals — acknowledging that a bar slightly before the exact extreme is still an excellent entry. Precise entry labels (`assign_entry_labels`) search a tight local window (up to 3 bars before and 4 after the swing, never past confirmation) and select the single best executable bar by a weighted score over risk-reward, ATR-scaled follow-through, wick structure, close location, proximity to the swing price, and timing:

$$score = 0.35 \cdot rr + 0.20 \cdot followthrough + 0.15 \cdot wick + 0.10 \cdot close\_loc + 0.10 \cdot proximity + 0.10 \cdot timing$$

A candidate is accepted only if its follow-through  $\geq 0.35$  ATR, its risk-reward  $\geq 1.25$ , and net progress toward the trade direction by the confirmation bar is positive. The breakout engine uses a structurally distinct entry scorer (`0.30·rr + 0.25·followthrough + 0.15·retest + 0.10·close_loc + 0.10·body + 0.10·timing`) with a 'hold' gate ensuring price does not fall back through the broken level. These entry labels are intentionally far sparser than zone labels.

Crucially, the engine does not treat every unlabeled bar as a clean negative. `create_exclusion_masks` builds purging zones around each swing confirmation (`exclusion_pre_bars = 10` bars before confirmation) and around positive zones, producing `exclude_long/exclude_short` masks; the complement that is neither positive nor excluded becomes the safe-negative set `neg_ok_long/neg_ok_short`. This prevents a classifier from being poisoned by ambiguous bars adjacent to unresolved structure. Finally, following the sample-uniqueness idea, the engine counts label concurrency and assigns inverse-concurrency sample weights, further multiplied by label quality, so that overlapping or low-quality positives contribute proportionally less:

$$w_i = (1 / \max(\text{concurrency}_i, 1)) \cdot \max(\text{quality}_i, \text{min\_label\_quality})$$

A CUSUM filter (`cusum_filter`) is also computed on structural-ATR-scaled log returns as a structural-break diagnostic, but — a deliberate and documented choice — it is not used as the final sampling universe; the swing engine defines the events.

### 3.7 The synthetic OTE target and class balance

The modeling targets `long_ote` and `short_ote` are synthetic unions defined in `preprocessing/config.py` and constructed in `feature_selection.py` (`SYNTHETIC_TARGET_COMPONENTS`). A bar is positive if any of its three direction-matched component labels (reversal, continuation-pullback, breakout) is positive; it is a clean negative only if it is not positive AND every component marks it as a safe negative AND no component excludes it; the sample weight is the row-wise maximum of the component weights. This union concentrates

three complementary structural setups into one detector while preserving each engine's exclusion discipline.

| Quantity                                  | long_ote                  | short_ote                 |
|-------------------------------------------|---------------------------|---------------------------|
| Usable rows (positives + clean negatives) | 443,926                   | 445,075                   |
| Positive rows                             | 24,810                    | 24,788                    |
| Clean-negative rows                       | 419,116                   | 420,287                   |
| Ambiguous rows excluded                   | 99,265                    | 98,116                    |
| Positive rate (modeling universe)         | 5.59%                     | 5.57%                     |
| Suggested positive class weight           | 16.89                     | 16.96                     |
| Train / Val / Test rows                   | 310,748 / 66,589 / 66,589 | 311,552 / 66,761 / 66,762 |

Table 3. Class balance of the synthetic OTE targets after ambiguity exclusion. Note the denominator: ~99k ambiguous bars are removed from the universe, so the modeling positive rate (~5.6%) is higher than the raw bar-level zone rate (~1.5%). Per-family bar-level positive rates in the full run were: reversal 1.56/1.55%, continuation 0.75/0.81%, breakout 2.28/2.23% (long/short).

The labeling discipline can be sanity-checked with an oracle backtest that trades the labels themselves (1.0×/2.0× ATR barriers, single position, conservative same-bar handling). On the reversal analysis set this oracle returns a 98.0% win rate and profit factor 137 over 11,011 trades. This figure is emphatically not a model result — it is an upper bound on label quality given perfect foresight of which bars are labeled, and it confirms only that the labels encode genuinely tradeable structure. Realistic, model-driven, cost-aware performance is reported in Section 7.

## 4. Feature Generation

Feature construction is handled by a modular builder (features/builder.py, FeatureDatasetBuilder) driven by JSON 'recipes'. A recipe declares an ordered list of feature sets plus a stack of post-feature transforms; the builder standardizes columns, validates OHLC, runs each registered feature set via a singleton FeatureRegistry, applies the transform blocks (optionally in parallel), drops warmup rows, forward-/zero-fills residual NaNs, optionally downcasts dtypes, and writes the dataset plus a metadata sidecar recording exact per-set column counts. The default OTE recipe (ote\_extended.json) produced 1,719 columns on the full run, of which 1,607 are engineered model features.

The 16 registered feature sets group naturally into the following categories:

| Category (feature set) | Representative features                                            | ~cols |
|------------------------|--------------------------------------------------------------------|-------|
| Price action           | close_return_1, candle_body, upper_wick, close_location, doji_like | 17    |
| Volatility             | atr_14, atr_ratio_14_50, rolling_vol_20, range_shock_20            | 11    |
| Trend                  | close_vs_ema_8_atr, ema_spread_21_50_atr,                          | 18    |

|                                    |                                                                                        |     |
|------------------------------------|----------------------------------------------------------------------------------------|-----|
|                                    | ema_alignment, macd_hist                                                               |     |
| <b>Momentum</b>                    | rsi_14, rsi_delta_3, roc_5,<br>price_acceleration_3                                    | 9   |
| <b>Volume</b>                      | log_volume, volume_relative_20,<br>volume_imbalance_10,<br>money_flow_10               | 8   |
| <b>Structure</b>                   | dist_to_prior_high_20_atr,<br>price_position_50, sweep_low_20,<br>displacement_bullish | 20  |
| <b>Higher-timeframe context</b>    | htf_30m_dist_to_swing_low_atr,<br>htf_1h_ema_alignment,<br>htf_alignment_score         | 33  |
| <b>ICT context</b>                 | dist_to_bull_fvg_atr,<br>dist_to_bear_order_block_atr,<br>ict_total_confluence_1atr    | 29  |
| <b>Continuation-pullback</b>       | pullback geometry, retest quality,<br>resumption strength                              | 22  |
| <b>Exhaustion</b>                  | return_slope_decay_3_8,<br>bullish_divergence_strength,<br>compression_regime          | 26  |
| <b>Microstructure</b>              | approx_spread, relative_spread_bps,<br>amihud_proxy, intrabar_efficiency               | 9   |
| <b>Session / calendar</b>          | hour_sin, dow_cos,<br>in_london_session,<br>in_london_ny_overlap, killzones            | 14  |
| <b>Temporal context</b>            | bars_since_last_sweep,<br>major_event_cluster_10, event<br>sequences                   | 24  |
| <b>Fractional differentiation</b>  | fracdiff_close, fracdiff_close_return_1                                                | 2   |
| <b>Quality</b>                     | anomaly_range_gt_10atr,<br>zero_volume_flag,<br>large_time_gap_flag                    | 6   |
| <b>Strategy signals (optional)</b> | one-hot/binary outputs of 400+ rule-<br>based strategies                               | 931 |

Table 4. The feature-set catalogue and per-set column counts from the full-run build. Indicator math uses pandas/NumPy and TA-Lib; ICT features encode fair-value-gaps, order blocks, liquidity pools, and break-of-structure as continuous distance/age features rather than binary flags.

On top of the feature sets, the recipe applies a transform stack that materially expands the space: lag features (174 columns; periods {1,2,3,5,10,20}), rolling mean/std statistics (160; windows {5,10,20,50,100}), rolling z-scores (18), rolling winsorization (13), rolling percentile ranks (10), ATR normalization (7), sigma normalization (17), and  $\approx 30$  curated interaction terms (e.g. trend $\times$ momentum alignment, session-conditioned setups, HTF-alignment interactions). Higher-timeframe features are built on completed right-closed/right-labeled bars and forward-filled to the

5-minute index so the model never sees an unfinished candle. Session features use America/New\_York as the feature clock with Asian (19:00–02:00), London (07:00–16:00 London), and New-York (08:00–17:00) sessions plus London (02:00–05:00) and New-York (08:00–11:00) kill-zones.

An optional `strategy_signals` family wraps a catalogue of 404 standalone rule-based strategy scripts adapted from public quantitative-trading sources. Each script consumes OHLCV and emits either categorical signals ('long'/'short'/'neutral'), binary buy/sell columns, or numeric indicator series; categorical outputs are one-hot encoded and every column is namespaced as `strategy__<id>__<column>`. Strategy execution is parallelized and sandboxed in subprocesses with per-strategy timeouts so a single faulty or slow strategy cannot stall or crash the build. As Section 5 shows, several of these strategy signals (notably Ichimoku state flags) survive all the way into the final feature rankings.

## 5. Feature Importance and Selection

With on the order of 1,600 engineered features and only ~5.6% positive bars, naive selection would overfit to noise. The pipeline therefore selects in two distinct layers: a target-aware base preprocessing stage that produces a clean, de-duplicated, collinearity-pruned feature pool and a composite base ranking, and a backend-aware attribution stage that re-ranks the survivors using the attribution method native to each model family before applying floor and cumulative-importance gates. Both stages are explicitly defended against look-ahead and noise.

### 5.1 Stage one: target-aware base preprocessing

`FeaturePreprocessingPipeline` (`features/preprocessing.py` with helpers in `preprocessing/feature_selection.py`) first resolves the eligible feature pool from builder metadata, falling back to a heuristic that excludes labels, helper columns, timestamps, and raw OHLCV. Candidate features are encoded (booleans to int, categoricals label-encoded with saved mappings, constant categoricals zeroed). It then removes exact duplicate columns via a blake2b hash signature with an equality confirmation (223 columns removed on the full run) and globally constant columns (139 removed), leaving 1,245 encoded features.

For each discovered target the pipeline builds a usable mask from the target label, the warmup mask, the direction-specific exclusion mask, and the safe-negative mask — so only positives and clean negatives enter the training universe. Missing-value fills are estimated on the training split only, choosing the median for strongly skewed columns ( $|\text{skew}| > 1$ ) and the mean otherwise, then applied identically to validation and test. Low-variance columns ( $\text{variance} \leq 1e-9$ ) are dropped, and a collinearity pruner removes one of each pair of features whose absolute correlation exceeds 0.98, keeping the member with the stronger association to the target; near-duplicate pairs above 0.995 are additionally recorded. After pruning, 1,052 (long) / 1,050 (short) features remain, with maximum residual pairwise correlation ~0.98.

The base ranking itself (`preprocessing/feature_importance.py`) is a rank-aggregation of three complementary signals computed on the most recent training rows, deliberately combining a linear, an information-theoretic, and a non-linear view so no single statistic dominates:

- **Association:** absolute Pearson correlation between each feature and the target (linear, fast, but blind to interactions).
- **Mutual information:** `mutual_info_classif` (captures non-linear and non-monotonic dependence).
- **RF importance:** a balanced-subsample RandomForestClassifier (250 trees, depth 8, min-leaf 8, OOB scoring) trained with the labeling sample weights; out-of-bag accuracy was 0.853 (long) / 0.858 (short).

Each signal is converted to a percentile rank and averaged into a `composite_score`. The diagnostics are sober and honest about difficulty: the maximum absolute association to the target is only ~0.255 and the maximum mutual information ~0.057 — no single feature is close to a silver bullet, which is exactly why the downstream models must combine many weak, regime-dependent features.

## 5.2 Stage two: backend-aware attribution

The key methodological innovation is that the tree model and the sequence models do not share a single feature ranking. `preprocessing/backend_attribution.py` trains a fast proxy model per backend on the top-160 base-ranked features and attributes importance using that backend's native method: TreeSHAP (`pred_contribs`) for the XGBoost proxy, and integrated gradients (16 interpolation steps from a zero baseline) for the TCN and LSTM proxies. For the sequence models the per-timestep attributions are summed across the window and read out at the latest bar, matching how the classifier head consumes the sequence. This produces per-feature mean-absolute-SHAP statistics computed separately over all rows and over positive rows.

The base composite rank and the attribution statistics are then merged into a single score (`merge_feature_rankings`). Each component is converted to a unit rank score and combined with weights `base = 0.20`, `SHAP-all = 0.55`, `SHAP-positive = 0.25` (re-normalized if a component is degenerate):

$$\text{merged\_score} = (0.20 \cdot \text{base\_rank} + 0.55 \cdot \text{shap\_all\_rank} + 0.25 \cdot \text{shap\_pos\_rank}) / \text{active\_weight\_sum}$$

The weighting deliberately privileges the model's own attribution (0.80 of the weight) over the model-agnostic base statistics, and the dedicated SHAP-positive term ensures features that matter specifically for the rare positive class are not drowned out by features that merely separate easy negatives. The merged ranking is then filtered by two sequential gates (`filter_features_by_attribution_gates`): a floor gate that drops any feature whose mean-absolute attribution is below a fraction of the maximum, and a cumulative gate that keeps only the highest-attribution features needed to reach a target cumulative share. The full run used `floor = 0.08` and `cumulative = 0.95` (the documented defaults are 0.15 / 0.90), with an automatic relaxation for rare targets (positive rate  $\leq 1.5\%$  tightens the cumulative gate to 0.98); the OTE targets at ~5.6% are not rare, so the requested gates apply directly.

The consequence is striking and is itself an important finding: the same 160 candidates collapse very differently per backend. For `long_ote`, the XGBoost proxy retains only 34 features (39 survive the floor, then the cumulative gate trims to 34) at a validation AP of 0.496 / ROC-AUC 0.921, whereas the TCN proxy retains 109 features at AP 0.874 / ROC-AUC 0.977 — the sequence model both spreads attribution across many more features and ranks the validation set far better. (These single-window validation APs are computed differently from the 8-fold event-matched CV/test APs in Section 6 and should not be compared directly.)

| Direction / backend | Method               | Candidates | Selected | Val AP | Val ROC-AUC |
|---------------------|----------------------|------------|----------|--------|-------------|
| long_ote / XGBoost  | TreeSHAP             | 160        | 34       | 0.496  | 0.921       |
| long_ote / TCN      | Integrated gradients | 160        | 109      | 0.874  | 0.977       |
| long_ote / LSTM     | Integrated gradients | 224        | 13       | 0.658  | 0.967       |
| short_ote / XGBoost | TreeSHAP             | 160        | 34       | 0.477  | 0.921       |
| short_ote / TCN     | Integrated gradients | 160        | 119      | 0.873  | 0.979       |

Table 5. Backend-aware attribution outcomes per target. Selected counts are the features surviving the floor + cumulative-importance gates; validation metrics are from the proxy models used only for attribution.

The surviving features are economically interpretable and remarkably consistent across backends. The recurring high-attribution drivers are: Ichimoku strategy state flags (long/short/neutral), distance to the prior 20-bar high/low in ATR units and their lags (dist\_to\_prior\_high\_20\_atr, dist\_to\_prior\_low\_20\_atr), EMA-distance features (close\_vs\_ema\_8\_atr, close\_vs\_ema\_21\_atr), ATR-normalized candle body (candle\_body\_atr\_norm), the 50-bar price position (price\_position\_50), and RSI. That distance-to-structure and trend-alignment features dominate is consistent with the labeling philosophy: OTE zones are defined relative to swing structure, so structural-distance features carry the most signal.

Two further details make the attribution stage robust. First, the SHAP/IG statistics are accumulated separately over all rows and over positive rows (mean\_abs\_shap\_all and mean\_abs\_shap\_positive), and the signed means are retained as well, so the merge can distinguish features that drive the rare positive class from those that merely shape the dominant-negative score surface — the 0.25 weight on the positive-class rank is precisely this safeguard. Second, the gate thresholds adapt to label rarity: when the development positive rate falls at or below 1.5%, the floor fraction is automatically tightened (to at most 0.05) and the cumulative-importance target is raised (to at least 0.98), so genuinely rare targets keep a wider feature set rather than being starved by an aggressive floor. The OTE targets, at ~5.6% positive, fall outside this rare-target branch and use the requested gates directly. The merged, filtered ranking is written to a backend-specific file (feature\_importance\_merged\_{xgboost,tcn,lstm}.csv) that the trainer automatically prefers over the generic ranking, closing the loop so that each model trains on the feature set its own attribution method endorsed.

### 5.3 Risks mitigated

The two-stage design targets the three classic ways feature selection fails in financial ML. Overfitting to noise features is mitigated by the floor and cumulative gates (which discard low-attribution features), by ranking on percentile aggregates rather than raw magnitudes, and by computing attribution on a held-out validation split rather than the training split. Look-ahead bias is mitigated by fitting all imputation, scaling, and ranking statistics on training data only and by inheriting the

causal construction of every feature. Multicollinearity is mitigated by the 0.98 correlation pruner that keeps the more target-associated member of each correlated pair. The trainer adds a final defense: a name-based leakage filter that blocks any feature whose name contains future, lookahead, pnl, mfe, mae, take\_profit, tp\_hit, sl\_hit, or exit\_signal.

## 6. Model Training and Hyperparameter Analysis

The trainer (`model_training/ote_training/ote_xgboost_pipeline.py`) consumes one prepared target at a time and supports three backends behind one data contract: XGBoost on sparse causal lag windows, and TCN or LSTM on dense causal windows of shape (samples, window, features). All three share the same purged walk-forward protocol, focal-loss objective, calibration, and event-level thresholding; only the temporal representation and inner optimizer differ.

### 6.1 Architectures

The XGBoost path builds a sparse lag view: it takes the top base features, a tuned window size, a tuned number of lag anchors within that window, and optionally delta features between the current bar and selected older lags — keeping tree training memory-efficient while preserving causal context. The TCN (`torch_models.py`, `TCNClassifier`) stacks residual blocks of two causal dilated 1-D convolutions (left-padded so no future bar leaks in) with GELU activations and exponentially growing dilation (1, 2, 4, ...); critically, the classifier head reads out the latest causal timestep rather than averaging the window, aligning the prediction with the current-bar label. The LSTM (`LSTMClassifier`) encodes the window and classifies from the final hidden state. Both sequence models use a 64-unit hidden size and 2–3 layers by default.

### 6.2 Training protocol and rare-event controls

Within each fold a `RobustScaler` is fit on training rows only (5th–95th percentile range, clipped to  $\pm 8$ ), avoiding leakage of future distribution. Model selection uses an expanding-window, purged walk-forward splitter: each fold has a growing training window, a purge/embargo gap, and a forward-only validation segment. The purge length is the maximum of the sequence-context length plus a 12-bar buffer and the event tolerance plus cooldown, which removes overlapping-label contamination; the logged CV used 8 folds with a 132-bar purge. Four imbalance controls operate simultaneously: a custom binary focal-loss objective (tunable  $\alpha$ ,  $\gamma$ ) that down-weights easy negatives; the labeling sample weights that preserve event uniqueness and quality; hard-negative up-weighting that emphasizes negatives near positives; and a balanced tuning subsample (negative ratio 8) during hyperparameter search.

Hyperparameter optimization is driven by Optuna with a TPE sampler and median pruning. Each fold contributes a weighted objective that aligns the search with both ranking quality and probability calibration:

$$\text{fold\_score} = 0.45 \cdot \text{average\_precision} + 0.45 \cdot \text{threshold\_score} + 0.10 \cdot (1 - \text{brier\_penalty})$$

where the `threshold_score` itself rewards event  $F\beta$  ( $\beta = 0.5$ ) and event precision while penalizing excess turnover (weights 0.55 / 0.45 / 0.40). Out-of-fold probabilities are then calibrated by Platt scaling (the production default), isotonic regression, or left raw. Finally the operating threshold is not fixed at 0.5: a 31-point grid is searched and the threshold maximizing event-level  $F0.5$  (with event

tolerance 2 bars, cooldown 4 bars) is selected, prioritizing event F0.5, then event precision, then closeness of predicted to true event count — an explicit alignment of thresholding to the trading use case.

The sequence backends follow an analogous discipline. Training proceeds through a phased learning-rate schedule — warmup, main, fine, and an optional tail — driven by AdamW with weight decay  $1e-4$ , gradient clipping at 1.0, ReduceLROnPlateau scheduling keyed to validation AUPRC (the rare-event-appropriate criterion), mixed-precision (AMP) when CUDA is available, and early stopping on validation performance. The fine and tail phases scale the learning rate down (fine\_lr\_scale 0.20–0.35, tail\_lr\_scale ~0.07–0.10) to settle the model without overfitting; the explicit four-phase schedule (e.g. 4/18/10/8 epochs summing to 40) is enforced to sum exactly to the requested epoch budget. The TCN and LSTM both consume RobustScaler-transformed dense windows built so that the label of the last window timestep is the bar being classified, with the training windows sliced to align the sequence end with the supervised bar. After cross-validation the best trial is refit on the combined development set and applied — with its learned scaler and calibrator — to the strictly held-out test split, exporting predictions, the Optuna trial log, training history, window-level feature importance, and a training summary.

### 6.3 The XGBoost hyperparameter search space

The tuned XGBoost space spans data-shape, regularization, focal-loss, schedule, and hard-negative parameters:

| Parameter           | Search range       | Parameter                    | Search range        |
|---------------------|--------------------|------------------------------|---------------------|
| window_size         | 8–40 (step 4)      | reg_alpha                    | $1e-4$ – 3.0 (log)  |
| lag_count           | 4–8                | reg_lambda                   | $1e-3$ – 10.0 (log) |
| delta_feature_count | 0–16               | min_split_loss               | 0.0 – 3.0           |
| learning_rate       | 0.015 – 0.12 (log) | max_delta_step               | 0.0 – 4.0           |
| focal_alpha         | 0.70 – 0.95        | max_bin                      | {256, 384, 512}     |
| focal_gamma         | 1.25 – 3.75        | warmup_rounds                | 20 – 80             |
| max_depth           | 3 – 7              | main_rounds                  | 120 – 420           |
| min_child_weight    | 1.0 – 14.0         | fine_rounds                  | 20 – 90             |
| subsample           | 0.65 – 1.0         | fine_lr_scale                | 0.15 – 0.6          |
| colsample_bytree    | 0.55 – 1.0         | hard_negative_radius / mult. | 1–8 / 1.0–2.5       |

Table 6. The Optuna search space for the XGBoost backend. Training is staged in three phases (warmup on an early subset, main on the full fold, fine-tune at reduced learning rate); the sequence backends use an analogous warmup/main/fine/tail epoch schedule.

### 6.4 Cross-validation and held-out results across model families

Table 7 reports the 8-fold purged-CV means and the held-out test metrics recorded in the model registry and candidate cohort. Several effects are clear and reproducible. First, the TCN backend is the strongest family on both ranking quality (AP) and event detection (event F0.5), on both sides.

Second, version-to-version retraining is not monotonic: the long TCN improved markedly from v1 to v2 (+0.064 test AP, +0.051 event F0.5) and the short TCN improved +0.081 test AP, but the long LSTM regressed catastrophically from v1 to v2 (−0.311 test AP) and XGBoost v2 was slightly worse than v1 on the long side — a reminder that re-tuning a sensitive sequence model can destroy as easily as it builds. Third, all families achieve very high ROC-AUC (~0.95–0.96) yet far lower AP (~0.55–0.76), the expected signature of a ~5.6% positive rate where AP is the honest metric and AUC is optimistic.

| Model (artifact)              | Backend | Win. | CV AP | CV ROC-AUC | CV ev.F0.5 | Test AP | Test ev.F0.5 |
|-------------------------------|---------|------|-------|------------|------------|---------|--------------|
| long_ote TCN v2               | TCN     | 20   | 0.625 | 0.961      | 0.774      | 0.763   | 0.852        |
| long_ote TCN v1 (champion)    | TCN     | 24   | 0.569 | 0.949      | 0.723      | 0.699   | 0.801        |
| long_ote XGB v1 (challenger)  | XGBoost | 8    | 0.548 | 0.962      | 0.707      | 0.667   | 0.704        |
| long_ote XGB v2               | XGBoost | 8    | 0.514 | 0.961      | 0.637      | 0.641   | 0.689        |
| long_ote LSTM v1 (benchmark)  | LSTM    | 28   | 0.519 | 0.947      | 0.691      | 0.635   | 0.797        |
| long_ote LSTM v2              | LSTM    | 20   | 0.277 | 0.916      | 0.434      | 0.324   | 0.504        |
| short_ote TCN v2 (champion)   | TCN     | 28   | 0.624 | 0.957      | 0.747      | 0.738   | 0.787        |
| short_ote TCN v1              | TCN     | 28   | 0.498 | 0.947      | 0.667      | 0.656   | 0.788        |
| short_ote XGB v1 (challenger) | XGBoost | 8    | 0.548 | 0.960      | 0.701      | 0.633   | 0.703        |
| short_ote XGB v2              | XGBoost | 8    | 0.554 | 0.959      | 0.685      | 0.620   | 0.720        |

Table 7. Purged 8-fold CV means (AP, ROC-AUC, event F0.5) and held-out test metrics per trained artifact. 'Win.' is the tuned sequence window length. The short-side LSTM was never trained. XGBoost final boosters used 460 trees (v1) and 430 trees (v2).

On the hyperparameter front, the strongest practical levers were the focal-loss focusing parameter and the tree complexity. Higher focal\_gamma concentrates gradient on the hard decision boundary and is essential at this imbalance; the search confined  $\alpha$  to 0.70–0.95 (strongly favoring the positive class) and  $\gamma$  to 1.25–3.75. Shallow trees (depth 3–7) with substantial min\_child\_weight (up to 14) and the three-phase warmup/main/fine schedule were the configuration that best controlled overfitting; the gap between CV and held-out AP stayed modest (e.g. long TCN v2 CV 0.625 → test 0.763, with

test actually higher because the held-out window happened to contain more detectable structure), and the catastrophic LSTM v2 regression (CV AP 0.277) is the clearest overfitting/instability indicator in the cohort — correctly caught by the evaluation stack and barred from promotion.

## 7. Model Testing, Regimes, and Backtesting

Offline model metrics are necessary but not sufficient. A model with high average precision can still be useless or dangerous if its skill is concentrated in regimes that rarely occur, if its threshold is mis-calibrated to the cost structure, or if its backtested edge evaporates under realistic frictions. This section describes the evaluation layer that turns trained probability models into promotion decisions: pairwise regime detection, regime-conditional evaluation, threshold calibration, and a cost-aware walk-forward backtest.

### 7.1 Pairwise regime detection

Regimes are computed deterministically from existing feature columns by `label_regimes` (`model_testing/ote_regime_labeler.py`). Four orthogonal regime axes are produced, and the central object is the pairwise composite regime defined as the Cartesian concatenation of the trend regime and the volatility regime:

$$\text{composite\_regime} = \text{trend\_regime} \oplus \text{"\_"} \oplus \text{vol\_regime}$$

The two constituent axes are constructed as follows:

- **Trend regime:** derived from ADX and EMA alignment.  $\text{ADX} < 20 \Rightarrow \text{ranging}$ ;  $20 \leq \text{ADX} \leq 30$  with a positive (negative) close-vs-EMA50 bias  $\Rightarrow \text{weak\_up}$  ( $\text{weak\_down}$ );  $\text{ADX} > 30$  with EMA alignment above +0.7 (below -0.7)  $\Rightarrow \text{strong\_up}$  ( $\text{strong\_down}$ ). Five states in total.
- **Volatility regime:** derived from the percentile rank of ATR over a trailing 504-bar ( $\approx$  one-week) window.  $\text{Rank} < 25 \Rightarrow \text{low}$ ;  $\text{rank} > 75 \Rightarrow \text{high}$ ; otherwise medium. Three states.

Their product yields up to 15 composite buckets (e.g. `strong_down_high`, `ranging_medium`, `strong_up_low`). Two further axes are computed but kept separate: a session regime (`asia` / `london` / `overlap` / `new_york` / `off_hours`) from the New-York-clock calendar, and a stress regime (`normal` / `elevated` / `high`) from the 20-bar range-shock feature ( $\geq 2$  elevated,  $> 3$  high).

Why pairwise rather than a single-feature regime or a latent-state model? The motivation is that EUR/USD behaviour is governed jointly by direction and energy, and the interaction is not separable. A strong downtrend in high volatility (`strong_down_high`) is a fundamentally different trading environment from a strong downtrend in low volatility, even though both share a trend label; the former offers large, clean swing reversals while the latter grinds. A single-variable trend regime would average these together and blur the very distinction that determines whether an OTE entry pays. Conditioning on the pair surfaces the interaction directly. The choice of a deterministic, rule-based pairwise regime over a Hidden Markov Model or other latent-state estimator is equally deliberate: the rule-based labels are causal, reproducible, auditable, and require no separate fitting or risk of look-ahead through the smoothing of a latent filter — properties that matter far more in a promotion-gate context than the marginal expressiveness an HMM might add. The empirical payoff is visible in the data: the long-side OTE positives concentrate overwhelmingly in down-trend×volatile buckets (`strong_down_medium` has 546 test positives, `strong_down_high` 459) while up-trend

buckets are thin (strong\_up\_low has 1), and short-side positives mirror this into up-trend buckets (strong\_up\_medium 525, strong\_up\_high 431). This asymmetry — long reversal entries cluster at the bottom of down moves, short entries at the top of up moves — is exactly what a pairwise regime makes legible and a single-axis regime would hide.

## 7.2 Regime-conditional evaluation

ote\_regime\_slices.py slices each model's predictions by every regime family (composite, trend, vol, session, stress, year) and computes, per bucket, the average precision with a bootstrapped confidence interval (200 resamples), the current-threshold event precision/recall/F0.5, and the per-bucket optimal threshold — but only where a bucket has at least 50 positive events, otherwise it falls back to the global threshold. A winner table then declares, per slice, which model leads and whether the win is 'confident' (the winner's AP confidence-interval lower bound exceeds the runner-up's upper bound). Event metrics use position-aware grouping: contiguous positive bars are collapsed into true zones, predictions are de-duplicated by a cooldown, and a prediction matches a zone if it falls within a 2-bar tolerance of it.

On the v1-vs-v2 comparison cohort (10 models, generated 2026-04-03) the regime slices are decisive. The long TCN v2 won all 27 evaluated long-side slices (11 of them confidently) and all 9 long-side composite buckets; the short TCN v2 won 26 of 27 short-side slices (12 confidently), losing only the thin ranging\_low bucket (18 positives) to XGBoost. The per-regime breakdown reveals exactly where skill lives:

| Regime slice (model)                       | AP    | Event F0.5 | Positives |
|--------------------------------------------|-------|------------|-----------|
| long TCN v2 • composite strong_down_high   | 0.915 | 0.925      | 459       |
| long TCN v2 • composite strong_down_medium | 0.719 | 0.828      | 546       |
| long TCN v2 • session overlap              | 0.919 | 0.927      | 271       |
| long TCN v2 • session london               | 0.851 | 0.908      | 359       |
| long TCN v2 • vol high                     | 0.889 | 0.912      | 564       |
| long TCN v2 • vol medium                   | 0.706 | 0.822      | 628       |
| short TCN v2 • session london              | 0.836 | 0.842      | 358       |
| short TCN v2 • trend strong_up             | 0.750 | 0.783      | 985       |
| short TCN v2 • stress normal               | 0.729 | 0.763      | 1,178     |
| short TCN v2 • vol low                     | 0.434 | 0.493      | 166       |
| short TCN v2 • composite strong_up_low     | 0.445 | 0.500      | 156       |

Table 8. Per-regime average precision and event F0.5 for the leading TCN models. Skill is highest in trend-aligned, high-volatility, London/overlap conditions and collapses in low-volatility and counter-trend-energy buckets (e.g. `vol low / strong_up_low` for the short model).

The lesson is unambiguous: the models are skilful precisely where the labeling philosophy predicts they should be — in volatile, trend-aligned, liquid-session conditions — and degrade sharply in low-volatility ranges where structure is faint. This is not a defect to be hidden but information to be exploited at the policy layer (Sections 7.3–7.4), which can lift thresholds or abstain in the weak regimes.

### 7.3 Threshold calibration

`ote_threshold_policy.py` searches operating thresholds that are regime-dependent and cost-aware. For each composite regime with at least 50 positive events it searches a 0.40–0.90 grid (step 0.05); thinner buckets inherit the global threshold. Candidate thresholds must clear a minimum event frequency (3 events/month), and the selection objective blends event quality with realized post-cost expectancy, evaluated under a leave-one-year-out scheme to discourage thresholds that overfit a single year:

$$\text{selection\_score} = 0.60 \cdot \text{event\_F0.5} + 0.40 \cdot \text{normalized\_post\_cost\_expectancy}$$

Costs are modeled explicitly. Each emitted trade pays the entry and exit spread (session-dependent: overlap 1.0, London/New-York 1.5, Asia 2.5, off-hours 3.0 pips), a slippage term of  $0.25 \times (\text{entry} + \text{exit spread})$ , plus a fixed slippage and commission (0.3 and 0.35 pips per trade in the backtest). The framework defines per-regime threshold hints (e.g. for long entries `strong_up_low` 0.55 but `ranging_low` 0.70 and `strong_down_high` abstain), encoding the prior that high-confidence is required in adverse regimes. Four policy variants are evaluated — `global_threshold`, `regime_threshold`, and their abstain-augmented counterparts.

The empirical result is a candid negative finding that the framework surfaces rather than hides. On the TCN focus cohort, only the long TCN v2 produced a regime-threshold policy that beat its own global-threshold baseline on both event F0.5 and post-cost expectancy, and even then the edge was tiny (event F0.5 +0.0012, expectancy +0.084 pips, net PnL actually –37 pips). The regime search mostly confirmed the global threshold (0.71), tightening only `strong_down_high` to 0.80. The hard-abstain variants behaved consistently and instructively: they sharply raised event F0.5 (the long TCN v2 jumped from 0.382 to 0.846) but slashed throughput (trades/week 11.8 → 4.1) and net PnL (14,573 → 4,469 pips). In other words, the current abstain configuration acts as a blunt trade suppressor — it makes the precision statistic look spectacular while destroying total profit — and was therefore explicitly not promoted.

Mechanically, the hard-abstain layer (`apply_abstain_policy`, `HardAbstainConfig`) sits downstream of the threshold and can veto a candidate signal for any of several reasons, each recorded for auditability: a high-stress regime, an off-hours session, membership in an explicitly blocked composite/session/stress regime (or blocked `composite×session` and `composite×stress` pairs), an expected move below a multiple (default 2.0×) of the session spread, a probability below a configurable quantile of the candidate distribution, and a cooldown that suppresses signals fired within 4 bars of an accepted one. The expected-move filter is the economically principled core — it abstains when the regime-conditional expected move does not clear the cost of crossing the spread

— but the breadth of the blanket high-stress and off-hours filters is what makes the current configuration too aggressive. The policy framework is sound; its parameterization is the open problem.

## 7.4 Backtesting framework

The promotion gate is an expanding-window, purged walk-forward backtest (`ote_policy_backtest.py`) that re-derives the entire policy fold by fold, so no future information from the threshold search leaks into earlier test windows. The procedure is:

1. Generate folds: a minimum 2-year training window, a 3-month forward test window, rolled forward in 3-month steps, with a 40-bar purge gap between train and test and at least 8 folds required. On the full history this yields 49 folds spanning 2012-10 to 2026-03 (13.5 years).
2. Within each fold, search regime thresholds on the training portion only (`search_thresholds_by_composite_regime`), producing a fold-local policy table and global threshold.
3. Select the policy variant (global, regime, or their abstain forms) that, on the training split, beats the global baseline on both event F0.5 and post-cost expectancy while maintaining  $\geq 3$  trades/week; if none qualifies, fall back to the global threshold.
4. Apply the selected policy to the untouched test window, attach forward trade outcomes (entry at the labeled bar close, exit 120 bars later) with full spread/slippage/commission costs, and record the emitted trades.
5. Aggregate test trades across all folds and compute performance: expectancy, profit factor, hit rate, max drawdown and its duration, monthly Sharpe and Sortino ( $\text{mean/std} \times \sqrt{12}$ ), trade concentration, and per-group breakdowns by year, quarter, session, composite regime, and confidence quintile.

Look-ahead is prevented at three levels: features and labels are causal by construction; folds are purged; and the policy itself is re-fit per fold on training data only. Walk-forward efficiency (WFE) — the ratio of mean test annualized PnL to mean train annualized PnL — measures generalization, and six acceptance gates (post-cost profitability, WFE above 0.5, profitable-quarter share, positive-composite-expectancy share, single-trade concentration, and drawdown versus average monthly profit) gate promotion.

## 7.5 Results across regimes and thresholds

Table 9 reports the cost-aware walk-forward backtest for the four TCN candidates over the full 49-fold history. All four are profitable after costs with WFE comfortably above 1.0 (i.e. they generalized better out-of-sample than in-sample on an annualized basis), and all four passed five of the six acceptance gates, failing only the strict 'max drawdown  $< 2\times$  average monthly profit' rule. The long TCN v2 is the standout:  $\approx 53,400$  net pips over 13.5 years at profit factor 1.96 and monthly Sharpe 2.36, and it is the only model for which the regime-threshold policy was selected in a large share of folds. The short TCN v2 materially improved on the short v1 — +50% net pips, higher profit factor (1.70 vs 1.43), higher Sharpe (1.90 vs 1.33), and roughly 25% lower drawdown — which is exactly why it was promoted to short-side champion.

| Model | Trades | Net pips | Exp. (pips) | Profit | Hit rate | Max DD | Mon. | WFE |
|-------|--------|----------|-------------|--------|----------|--------|------|-----|
|-------|--------|----------|-------------|--------|----------|--------|------|-----|

|              |       |        |      | factor |       | (pips) | Sharpe |      |
|--------------|-------|--------|------|--------|-------|--------|--------|------|
| long TCN v2  | 6,514 | 53,379 | 8.20 | 1.96   | 60.6% | 1,977  | 2.36   | 1.98 |
| long TCN v1  | 5,140 | 39,585 | 7.70 | 1.91   | 60.6% | 1,805  | 2.23   | 1.93 |
| short TCN v2 | 5,160 | 34,079 | 6.60 | 1.70   | 59.3% | 4,068  | 1.90   | 1.66 |
| short TCN v1 | 5,182 | 22,925 | 4.42 | 1.43   | 57.4% | 5,433  | 1.33   | 8.57 |

Table 9. Cost-aware walk-forward backtest (49 folds, 2012-10 to 2026-03) for the TCN candidate cohort, with 0.3-pip slippage and 0.35-pip commission per trade plus session spreads. The short TCN v1's inflated WFE (8.57) reflects a near-zero in-sample denominator, not superior generalization — a reminder to read WFE alongside absolute PnL.

Three cross-cutting conclusions emerge from the regime/threshold/model grid. (1) Model family dominates everything else: across every slice and the full backtest, the TCN beats XGBoost and LSTM, and the gap is largest exactly in the high-skill regimes. (2) Regime-conditional thresholds help only marginally over a well-chosen global threshold for these models, because the global threshold already sits near the regime-blended optimum; their value is concentrated in the few data-rich adverse buckets (e.g. tightening strong\_down\_high). (3) The largest available lever is not the threshold grid but the abstain logic, which is currently mis-tuned — it improves precision but suppresses profit — and is the clearest target for future work. The active production registry reflects these findings: the long champion is a TCN (test AP 0.699, event F0.5 0.801) carrying a full nine-bucket regime-threshold map, the short champion is the TCN v2 (test AP 0.738) on a global 0.75 threshold, with XGBoost retained as challenger and LSTM as a diversity benchmark; no abstain policy is written into any entry.

## 7.6 Confidence calibration and aggregate ranking

Beyond regime and threshold, the backtest stratifies test trades into confidence quintiles by calibrated probability and reports per-quintile expectancy, providing a direct check on probability calibration: a well-calibrated model should show monotonically rising expectancy with confidence, and the Platt-scaled probabilities used here are fit on out-of-fold predictions precisely to support this. Drawdown is measured on the per-trade equity curve as the maximum peak-to-trough deviation in pips together with its underwater duration in days, and trade concentration is summarized by a Herfindahl index and the largest single trade's share of total P&L — the long TCN's largest trade contributes under 0.8% of total profit, confirming that the edge is broad rather than driven by a few outliers. When the framework's many artifacts are collapsed into a single cross-model leaderboard (across 40+ training artifacts and dozens of backtests), the OTE-full TCN models rank highly on post-cost Sharpe and profit factor, but the very top of the post-training leaderboard is occupied by a narrower reversal-family model, while the training-only leaderboard is dominated by breakout models whose high in-sample average precision does not convert to post-cost robustness. That gap between training rank and post-cost rank is itself the strongest argument for the entire evaluation stack: the metric that selects the champion must be the one that survives costs and time, not the one that looks best in cross-validation.

## 8. Discussion

What worked. The combination of structural-ATR thresholds, triple-barrier outcome validation, and trend-scan confirmation produced labels that an oracle could trade at a 98% win rate — a strong upper bound — and that real models could detect at test average precisions of 0.70–0.76, well above the ~0.06 base rate. The TCN architecture, reading out the latest causal timestep, was consistently the best learner of this structure, and the event-level evaluation and cost-aware walk-forward backtest translated that statistical skill into a post-cost edge that survived 13.5 years and 49 folds. The pairwise regime layer earned its keep as an explanatory and diagnostic tool, cleanly localizing skill to trend-aligned, volatile, liquid sessions.

What did not work, and what surprised us. The hard-abstain policy was the clearest negative result: it is seductive because it dramatically improves the precision statistics that practitioners instinctively trust, yet it destroyed total profitability by suppressing too many trades. This is a cautionary tale about optimizing the wrong objective — precision is not P&L. Regime-conditional thresholds, despite strong a priori appeal, added little over a good global threshold, because the global optimum already integrates over the regime mix. Version retraining was non-monotonic and occasionally destructive (LSTM v2), underscoring that sequence models on noisy financial data are fragile and that an evaluation stack which can veto a regression is as valuable as one that can crown a champion. We were also reminded that headline AUC (~0.96) badly overstates usefulness at a 5.6% base rate; average precision and event F0.5 are the metrics that track reality.

Practical implications for the financial ML community. The most transferable ideas here are architectural rather than parameter-specific: label by validated structure rather than fixed horizons; express thresholds in volatility units; treat ambiguous bars as exclusions, not negatives; weight by event uniqueness; select features with the model's own attribution method; evaluate and threshold at the event level; condition everything on a pairwise regime; and never promote without a purged, cost-aware walk-forward gate. Each of these directly attacks a known failure mode of naive financial ML.

### 8.1 Limitations

- The 98%-win-rate label backtest is an oracle, not a tradeable result; it bounds label quality only.
- Prepared split CSVs drop timestamps, so downstream joins are row-index keyed; the regime/threshold result artifacts persist primarily as summary documents in this snapshot rather than as raw CSVs.
- The short-side LSTM was never trained, so the LSTM family is benchmarked only on the long side.
- Single-window validation AP (used for attribution proxies) and 8-fold event-matched CV/test AP are computed differently and must not be compared directly; likewise the policy-evaluation event F0.5 (measured over de-duplicated emissions across the whole test set) is not the same denominator as the per-regime slice event F0.5.
- Backtest entries assume execution at the labeled bar close and a fixed 120-bar exit; richer execution and exit logic, and partial fills, are not modeled.
- CUSUM events are computed but unused as a sampler; the abstain policy is mis-tuned; and results are specific to EUR/USD 5-minute data over 2012–2026 and should not be assumed to transfer unchanged to other pairs, timeframes, or future regimes.

## 9. Conclusion

We have documented an end-to-end, causally disciplined machine-learning framework for EUR/USD optimal-trade-entry detection that spans adaptive multi-engine labeling, broad but defensible feature engineering, two-stage backend-aware feature selection, purged walk-forward model training across three architectures, and a regime-aware evaluation and backtesting layer. The framework's contributions are an adaptive structural-ATR labeling methodology with triple-barrier and trend-scan validation; a backend-specific attribution-merge selection stage; a pairwise trend×volatility regime construction for conditional evaluation; and a cost-aware, leave-one-year-out threshold search feeding a 49-fold walk-forward promotion gate. Empirically, TCN models lead on both sides (test AP 0.74–0.76, event F0.5 0.79–0.85) and remain profitable after realistic costs over 13.5 years (long ≈53k net pips, profit factor 1.96, Sharpe 2.36). Future work should focus on the framework's own diagnosed weaknesses: re-designing the abstain policy as a profit-aware risk filter rather than a precision-maximizing suppressor; stabilizing sequence-model retraining; extending the regime construction with the still-separate session and stress axes; modeling richer execution and dynamic exits; and validating cross-pair and forward-walk transfer. The broader message is methodological: in financial machine learning, the model is the easy part — disciplined labeling, honest event-level evaluation, regime conditioning, and cost-aware walk-forward validation are what separate a flattering backtest from a deployable system.

## Appendix A. Reversal Labeling Default Parameters

| Parameter              | Value           | Parameter                   | Value |
|------------------------|-----------------|-----------------------------|-------|
| atr_period             | 14 (Wilder/EMA) | tb_profit_atr               | 1.0   |
| structural_atr_tf      | 1hr             | tb_stop_atr                 | 2.0   |
| cusum_atr_mult         | 0.5             | tb_maxBars                  | 120   |
| confirm_atr_mult       | 0.8             | zone_preBars / postBars     | 2 / 0 |
| min_swing_atr          | 1.35            | entry_lookback / max_delay  | 3 / 4 |
| min_swing_distance_atr | 0.75            | entry_min_rr                | 1.25  |
| minBars_betweenSwings  | 18              | entry_min_followthrough_atr | 0.35  |
| trend_scan_windows     | (12,24,48,96)   | trend_scan_min_abs_t        | 2.0   |
| min_label_quality      | 0.35            | exclusion_preBars           | 10    |
| htf_confluence_window  | 120 min         | warmupBars                  | 50    |

Table A1. ReversalParams defaults (build\_default\_params). The continuation and breakout engines subclass these with 30-minute structural ATR and family-specific thresholds (Section 3.4, Table 2).

## Appendix B. Evaluation and Backtest Configuration

| Setting | Value | Setting | Value |
|---------|-------|---------|-------|
|---------|-------|---------|-------|

|                                |                                            |                                   |                             |
|--------------------------------|--------------------------------------------|-----------------------------------|-----------------------------|
| Regime ADX ranging / strong    | 20 / 30                                    | Backtest min train years          | 2                           |
| EMA-alignment strong threshold | $\pm 0.7$                                  | Test window / step                | 3 mo / 3 mo                 |
| ATR percentile window          | 504 bars                                   | Purge gap (backtest)              | 40 bars                     |
| Vol low / high percentile      | 25 / 75                                    | Min folds (achieved)              | 8 (49)                      |
| Threshold grid                 | 0.40–0.90 (0.05)                           | Event tolerance / cooldown        | 2 / 4 bars                  |
| Threshold objective            | $0.60 \cdot F0.5 + 0.40 \cdot \text{exp.}$ | Min positive events (regime thr.) | 50                          |
| Session spreads (pips)         | ovlp1.0/Ln1.5/NY1.5/As2.5/off3.0           | Slippage multiplier               | $0.25 \times \text{spread}$ |
| Fixed slippage / commission    | 0.3 / 0.35 pips                            | Bootstrap AP iterations           | 200                         |
| Min events/month, trades/week  | 3.0 / 3.0                                  | Confidence buckets                | 5 quintiles                 |

Table B1. Key configuration constants for the regime labeler, threshold search, and walk-forward backtest.

## Appendix C. Active Model Registry

| model_id                   | Dir   | Role       | Backend | Test AP | Test ev.F0.5 | Global thr.        |
|----------------------------|-------|------------|---------|---------|--------------|--------------------|
| long_ote_champion_v1       | long  | champion   | TCN     | 0.699   | 0.801        | 0.75 (+regime map) |
| long_ote_challenger_v1     | long  | challenger | XGBoost | 0.667   | 0.704        | 0.68               |
| short_ote_candidate_tcn_v2 | short | champion   | TCN     | 0.738   | 0.787        | 0.75               |
| short_ote_candidate_xgb_v1 | short | challenger | XGBoost | 0.633   | 0.703        | 0.68               |
| long_ote_benchmark_lstm_v1 | long  | benchmark  | LSTM    | 0.635   | 0.797        | 0.50               |

Table C1. The active model registry (models/ote\_model\_registry.json). Promotion rules require  $\geq 3$  CV splits, test event  $F0.5 \geq 0.65$ , regime robustness, post-cost profitability, and paper-trading confirmation. The long champion carries a 9-bucket regime-threshold map (ranging 0.68; strong\_down\_high 0.80, strong\_down\_medium 0.70; strong\_up\_low 0.75; others 0.68).

## Appendix D. Selected References (Methodological Lineage)

The labeling engine cites and operationalizes the following concepts from M. López de Prado, *Advances in Financial Machine Learning* (Wiley, 2018):

- CUSUM filter for event-driven sampling (Ch. 2.5.2) — computed as a structural-break diagnostic.
- Dynamic, volatility-adaptive thresholds (Ch. 3.3) — realized as structural-ATR scaling.
- The triple-barrier method for outcome-based label validation (Ch. 3.4).
- Exclusion zones / purging to prevent look-ahead contamination (Ch. 7.1).
- Sample uniqueness and concurrency-based weighting (Ch. 4.4–4.5).
- Meta-labeling readiness for a downstream secondary model (Ch. 3.6).

Trend-scan labeling follows the forward-looking trend t-statistic approach in the same lineage.

Technical indicators are computed with TA-Lib and the `ta` library; the rule-based strategy catalogue is adapted from the QuantifiedStrategies corpus. Gradient-boosted trees use XGBoost; sequence models use PyTorch; feature attribution uses TreeSHAP and integrated gradients.